

Random Forest: Predicting Breast Cancer Diagnosis

Scenario

Predict whether a breast tumour is malignant (Cancerous) or benign (Non-Cancerous). This project applies a Random Forest classifier to predict whether a breast tumour is malignant (cancerous) or benign (non-cancerous) based on diagnostic cell features. Accurate predictions can help medical practitioners make informed decisions and improve patient outcomes.

Dataset Overview

The dataset contains diagnostic features of breast tumours along with the target variable indicating whether a tumour is malignant (1) or benign (0). The data description is as follows:

- **Clump Thickness:** Measures the layering of cells in a tumour. Benign cells form single layers, while cancerous cells form multiple layers.
- **Uniformity of Cell Size/Shape:** Assesses how similar cells are in size and shape under a microscope.
- **Marginal Adhesion:** Indicates how well cells stick together; poor adhesion may signal malignancy.
- **Single Epithelial Cell Size:** Refers to the size of individual epithelial cells, which form tightly packed layers covering surfaces and lining organs.
- **Bare Nuclei:** Identifies cells with only the nucleus visible, missing membranes and cytoplasm.
- **Bland Chromatin:** Describes compact, uniform genetic material in the nucleus, typical of normal cells.
- **Normal Nucleoli:** Small, well-defined structures in the nucleus that produce ribosomes; they appear distinct in healthy cells.
- **Mitosis:** The process of cell division, essential for tissue growth, repair, and new cell production.

Columns:

- **Input Variables (Features):**
 - Clump Thickness
 - Uniformity of Cell Size
 - Uniformity of Cell Shape
 - Marginal Adhesion
 - Single Epithelial Cell Size
 - Bare Nuclei
 - Bland Chromatin
 - Normal Nucleoli
 - Mitoses
- **Target Variable:**
 - Class (0 = Benign, 1 = Malignant)

Why Random Forest?

Random Forest is suitable for this scenario because:

1. It combines the predictions of multiple decision trees, reducing overfitting and improving generalisation.
2. It handles high-dimensional data and can identify the most important features for classification.
3. It is robust to noise and missing data, making it ideal for real-world applications.
4. It provides higher accuracy and stability compared to individual decision trees.

Steps to Implement Random Forest in Python

Step 1: Import Libraries

```
In [1]: ▶ import pandas as pd # For data manipulation and analysis
from sklearn.model_selection import train_test_split # For splitting the
from sklearn.ensemble import RandomForestClassifier # Random Forest model
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt # Matplotlib
import seaborn as sns # seaborn
```

Step 2: Load the Dataset

```
In [2]: ▶ # Load the dataset
file_path = 'breast_cancer.csv'
data = pd.read_csv(file_path)

# Display the first few rows
print(data.head())
```

	Clump Thickness	Uniformity of Cell Size	Uniformity of Cell Shape	\
0	5	1	1	
1	5	4	4	
2	3	1	1	
3	6	8	8	
4	4	1	1	

	Marginal Adhesion	Single Epithelial Cell Size	Bare Nuclei	\
0	1	2	1	
1	5	7	10	
2	1	2	2	
3	1	3	4	
4	3	2	1	

	Bland Chromatin	Normal Nucleoli	Mitoses	Class
0	3	1	1	0
1	3	2	1	0
2	3	1	1	0
3	3	7	1	0
4	3	1	1	0

Step 3: Data Overview

```
In [3]: # Structure , data type, summary statistics  
data.shape  
data.info()  
data.describe()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 683 entries, 0 to 682  
Data columns (total 10 columns):  
#   Column                                Non-Null Count  Dtype  
---  -  
0   Clump Thickness                       683 non-null    int64  
1   Uniformity of Cell Size               683 non-null    int64  
2   Uniformity of Cell Shape              683 non-null    int64  
3   Marginal Adhesion                     683 non-null    int64  
4   Single Epithelial Cell Size           683 non-null    int64  
5   Bare Nuclei                           683 non-null    int64  
6   Bland Chromatin                       683 non-null    int64  
7   Normal Nucleoli                       683 non-null    int64  
8   Mitoses                               683 non-null    int64  
9   Class                                 683 non-null    int64  
dtypes: int64(10)  
memory usage: 53.5 KB
```

Out[3]:

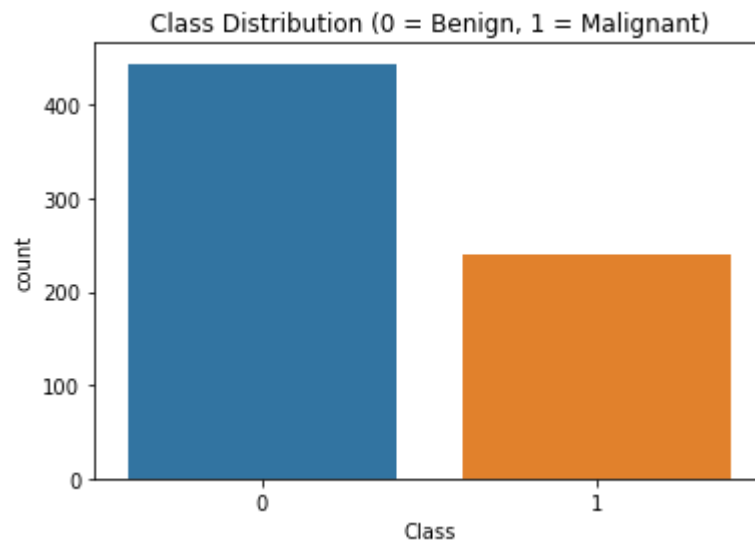
	Clump Thickness	Uniformity of Cell Size	Uniformity of Cell Shape	Marginal Adhesion	Single Epithelial Cell Size	Bare Nuclei	Bland Chromatin
count	683.000000	683.000000	683.000000	683.000000	683.000000	683.000000	683.000000
mean	4.442167	3.150805	3.215227	2.830161	3.234261	3.544656	3.445095
std	2.820761	3.065145	2.988581	2.864562	2.223085	3.643857	2.449697
min	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
25%	2.000000	1.000000	1.000000	1.000000	2.000000	1.000000	2.000000
50%	4.000000	1.000000	1.000000	1.000000	2.000000	1.000000	3.000000
75%	6.000000	5.000000	5.000000	4.000000	4.000000	6.000000	5.000000
max	10.000000	10.000000	10.000000	10.000000	10.000000	10.000000	10.000000



Step 4: Class Distribution

```
In [4]: ▶ # Class Distribution
        # Checking balance between benign and malignant cases.

sns.countplot(x='Class', data=data)
plt.title("Class Distribution (0 = Benign, 1 = Malignant)")
plt.show()
```



Step 5: Data Preparation

```
In [5]: ▶ # Separate features and target variable
        X = data.drop(columns=['Class']) # Features (all except 'Class')
        y = data['Class'] # Target variable (Class: 0 = Benign, 1 = Malignant)

        # Split the dataset into training and testing sets
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, r
```

- **Input Variables (X):** All numeric features from the dataset.
- **Target Variable (y):** Diagnosis (0 for Benign, 1 for Malignant).

Step 6: Train the Model

```
In [6]: ► # Initialise and train the random forest model
model = RandomForestClassifier(random_state=42, n_estimators=100, max_depth=10)
model.fit(X_train, y_train)
```

```
Out[6]: RandomForestClassifier(max_depth=10, random_state=42)
```

Explanation

Initialising the Model:

- `RandomForestClassifier(random_state=42, n_estimators=100, max_depth=10)` :
 - `random_state=42` : Ensures reproducibility by fixing the randomness in the model training process.
 - `n_estimators=100` : Specifies the number of decision trees in the forest.
 - `max_depth=10` : Limits the depth of each tree to prevent overfitting.

Training the Model:

- `model.fit(X_train, y_train)` :
 - The model learns from the training data by building multiple decision trees and aggregating their predictions to provide a more accurate and robust classification.

Step 7: Evaluate the Model

```
In [7]: ▶ # Make predictions
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

# Confusion matrix and classification report
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Accuracy: 0.95

Confusion Matrix:

```
[[125  2]
 [ 8 70]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.94	0.98	0.96	127
1	0.97	0.90	0.93	78
accuracy			0.95	205
macro avg	0.96	0.94	0.95	205
weighted avg	0.95	0.95	0.95	205

Evaluation Metrics

Key Metrics:

1. **Accuracy Score:** Proportion of correctly classified samples.
2. **Confusion Matrix:** Breakdown of true positives, true negatives, false positives, and false negatives.
3. **Classification Report:** Includes precision, recall, F1-score, and support for each class.

Results Interpretation:

- **Accuracy:** The model achieved an accuracy of **95%**, meaning 95% of the samples were classified correctly.

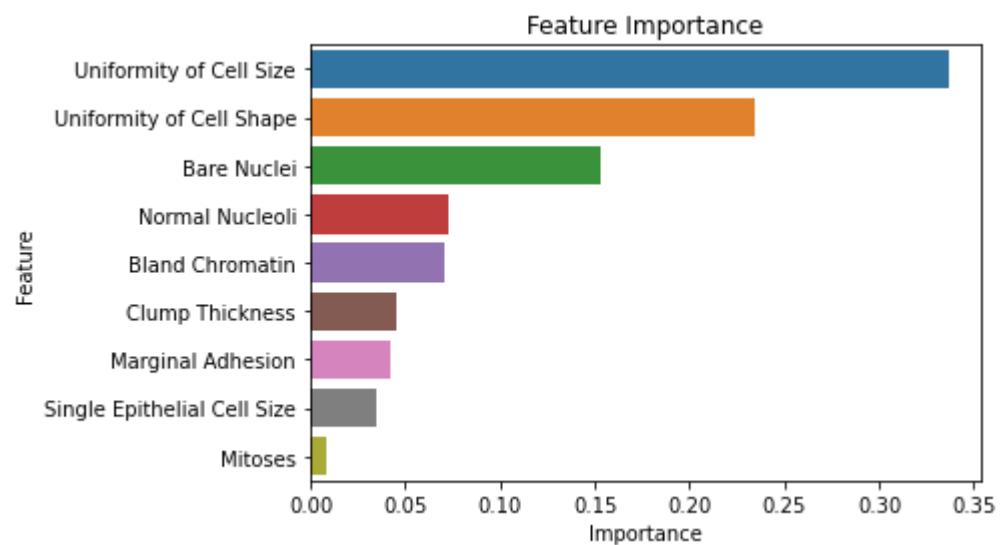
Step 8: Feature Importance

```
In [8]: ▶ # Feature importance

importances = model.feature_importances_
features = X.columns

feat_df = pd.DataFrame({
    'Feature': features,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

sns.barplot(x='Importance', y='Feature', data=feat_df)
plt.title("Feature Importance")
plt.show()
```



Step 9: Make Predictions with New Data

```
In [9]: ▶ # Example new data for prediction
new_data = pd.DataFrame({
    'Clump Thickness': [5, 8],
    'Uniformity of Cell Size': [1, 10],
    'Uniformity of Cell Shape': [1, 8],
    'Marginal Adhesion': [1, 7],
    'Single Epithelial Cell Size': [2, 5],
    'Bare Nuclei': [1, 10],
    'Bland Chromatin': [3, 8],
    'Normal Nucleoli': [1, 9],
    'Mitoses': [1, 2]
})

# Predict the diagnosis for new data
predicted_classes = model.predict(new_data)
print("Predicted Classes:", predicted_classes)
```

Predicted Classes: [0 1]

```
In [10]: ▶ data.to_csv('breastcancer.csv')
```

Interpretation of Predicted Classes

The output Predicted Classes: [0 1] represents the predictions made by the Random Forest model for the given input data:

- **0:** The model predicts the first sample as benign (non-cancerous).
- **1:** The model predicts the second sample as malignant (cancerous).

These predictions are based on the Random Forest model, which combines the outputs of multiple decision trees to determine the most likely class for each sample.

Conclusion

Random Forest is a robust and powerful classification model that combines the predictions of multiple decision trees to improve accuracy and stability. By aggregating the results of many trees, it reduces overfitting and enhances generalisation.

In this case, the Random Forest model achieved an accuracy of **95%**, which is higher than the **94% accuracy** achieved by the Decision Tree model. This demonstrates the effectiveness of Random Forest in improving classification performance. Its ability to handle complex data and identify the most important features makes it a valuable tool for healthcare applications.

